

TRƯỜNG ĐẠI HỌC SƯ PHẠM – ĐẠI HỌC ĐÀ NẴNG
KHOA TOÁN – TIN

Học phần: Lập trình mạng (Network Programming) – Học kỳ I, năm học 2026 – 2027

BÀI TẬP THỰC HÀNH NHÓM SỐ 3

Chương 3 – Lập trình ứng dụng hướng kết nối TCP

Xây dựng server chat tuần tự hoàn chỉnh, client dòng lệnh và mô-đun protocol.py dùng chung

Hình thức	Làm việc nhóm 5 sinh viên — giữ nguyên nhóm đã đăng ký ở Bài tập nhóm số 1. Bài làm phát triển tiếp bước đăng nhập mà nhóm đã cài đặt ở Bài tập nhóm số 2.
Thời hạn	Nộp trên e-Learning (nhhai.net) trước 21:00 thứ Năm, ngày 01/10/2026.
Sản phẩm nộp	Một báo cáo PDF và mã nguồn Python (không phải làm slide, không thuyết trình). Tên file bắt buộc phải bắt đầu với NhomZZ, trong đó ZZ là mã nhóm. Ví dụ: Nhom05_Baitap3.pdf
Chuẩn đầu ra	CLO2 – giải thích và vận dụng được lập trình ứng dụng hướng kết nối TCP: vòng đời kết nối, server tuần tự, đặc tả giao thức, ranh giới và định dạng thông điệp. CLO5 – tổ chức làm việc nhóm, tìm kiếm tài liệu tham khảo có giá trị. CLO7 – có ý thức đóng góp để tạo ra một chương trình hoàn chỉnh.
Công cụ	Python 3.12 trở lên (không cài thêm thư viện ngoài); Visual Studio Code (extension Python) hoặc PyCharm Community; nc, nc hoặc telnet; netstat hoặc lsof.

1. Mục tiêu

Sau khi hoàn thành bài tập, sinh viên có thể:

- Xây dựng được một TCP server tuần tự hoàn chỉnh theo bảy bước đã học: đặc tả giao thức, mở cổng, vòng accept, vòng hội thoại, xử lý một yêu cầu, xử lý lỗi và dọn dẹp, cấu hình và ghi log.
- Viết được client dòng lệnh đi kèm: giữ kết nối bền vững suốt phiên, đọc đúng câu trả lời nhiều dòng và báo lỗi dễ hiểu cho người dùng trong mọi tình huống hỏng kết nối.
- Giải thích và chứng minh được bằng số liệu chạy thật hiện tượng dính gói – tách gói của TCP; chọn và cài đặt đúng cách đóng khung thông điệp cho từng loại dữ liệu (dòng văn bản và nội dung tệp nhị phân).
- Tự kiểm thử chương trình của mình bằng một đoạn mã lặp lại được, trong đó có các ca dữ liệu sai, thay vì chỉ chạy thử bằng tay một lần.

2. Nhiệm vụ

Quy ước chung: mỗi nhóm vẫn dùng cổng **9000 + mã nhóm** (nhóm 05 dùng cổng 9005). Mọi chương trình phải nhận `--host` và `--port` từ dòng lệnh bằng `argparse`; mặc định server là `0.0.0.0`, client là `127.0.0.1` để giảng viên chạy thử được cả hai chương trình trên một máy mà không phải sửa mã.

2.1. Phần A – Server chat tuân tự `chat_server.py` (3,5 điểm)

Nhóm hoàn thiện đặc tả giao thức chat của mình rồi cài đặt server tuân tự phục vụ đủ bốn lệnh `LOGIN`, `MSG`, `HISTORY` và `QUIT`. Khung mã có trong slide Chương 3, mục 3.2.

- Bản đặc tả giao thức.** Cập nhật bảng đặc tả của nhóm (từ Bài tập nhóm số 1 và 2) cho đủ bốn lệnh theo mẫu ở Bảng 1, kèm bảng mã lỗi ít nhất năm mã và sơ đồ ba trạng thái phiên (chưa đăng nhập – đã đăng nhập – kết thúc). Ghi rõ ranh giới thông điệp, ký tự phân cách, tên dành riêng và bên nào đóng kết nối trước.
- Tổ chức mã thành hàm.** Server phải tách thành các hàm `main()`, `create_listener()`, `serve_forever()`, `handle_client()` và `handle_request(line, sess)`. Hai hàm `send_line()` và `recv_line()` đặt trong tệp dùng chung `protocol.py` (có giới hạn `MAX_LINE` và `rstrip("\r")`), cả server lẫn client cùng import.
- Bốn lệnh và dữ liệu dùng chung.** Kiểm tra tên đăng nhập đủ bốn điều (rỗng, quá 20 ký tự, chứa ký tự phân cách, trùng tên hoặc tên dành riêng); `MSG|all|<nội dung>` lưu tin vào bảng tin chung; `HISTORY|<n>` trả lời nhiều dòng với dòng đầu báo trước số dòng còn lại; `QUIT` trả `BYE` rồi đóng phiên. Lệnh sai trình tự hoặc sai cú pháp phải trả mã lỗi, không được ném ngoại lệ.
- Cấu hình, log và dừng an toàn.** Dùng `argparse` cho `--host` và `--port`; dùng `logging` có mốc thời gian để ghi lại việc mở cổng, mỗi kết nối mới, mỗi dòng nhận được và lúc đóng phiên; `Ctrl+C` dừng được server. Lỗi của một client không được làm chết server: phải bắt lỗi trong `handle_client()` và luôn xóa tên khỏi danh sách đang trực tuyến khi phiên kết thúc, kể cả khi client rút dây.
- Chạy thử bằng công cụ có sẵn.** Dùng `ncat` (hoặc `nc`, `telnet`) gõ tay một phiên đầy đủ `LOGIN` → `MSG` → `HISTORY` → `QUIT` và một phiên có ít nhất hai lệnh sai. Chụp màn hình cả cửa sổ gõ tay lẫn cửa sổ log của server, và chép nội dung log vào báo cáo dưới dạng văn bản.

Bảng 1. Mẫu bảng đặc tả thông điệp (dòng đầu là ví dụ, nhóm thay bằng đặc tả của mình)

Lệnh	Cú pháp gửi đi	Điều kiện hợp lệ	Trả lời khi đúng	Trả lời khi sai
LOGI N	LOGIN <tên>\n	Chưa đăng nhập; tên 1 – 20 ký tự, không chứa " "	OK Chào <tên>\n	ERR 1, ERR 2, ERR 3 (xem bảng mã lỗi)
...	...	...	...	...

2.2. Phần B – Client `chat_client.py` và kiểm thử `test_server.py` (3,0 điểm)

Client giữ một kết nối bền vững suốt phiên làm việc; bộ kiểm thử chạy được bằng một lệnh và tự báo Đạt / Sai cho từng ca.

1. **Vòng lặp hỏi – đáp.** Client mở kết nối bằng `socket.create_connection()`, đặt `settimeout()`, rồi lặp: nhập lệnh từ bàn phím → `send_line()` → đọc và in câu trả lời → thoát khi nhận BYE. Bộ đệm buf của `recv_line()` phải được giữ và truyền lại suốt phiên, không tạo mới sau mỗi lệnh.
2. **Đọc đúng câu trả lời nhiều dòng.** Khi dòng đầu là `HISTORY|<n>`, client đọc đúng n dòng tiếp theo — không thừa, không thiếu. Giải thích trong 3 – 5 dòng điều gì xảy ra nếu client đọc thừa một dòng hoặc thiếu một dòng.
3. **Báo lỗi cho người dùng.** Bắt riêng `ConnectionRefusedError`, `TimeoutError`, `ConnectionError` và `EOFError` / `KeyboardInterrupt`; mỗi trường hợp in một câu tiếng Việt cho biết nên làm gì. Lập bảng: tình huống – cách tạo ra tình huống đó – thông báo chương trình in ra.
4. **Bộ kiểm thử `test_server.py`.** Ít nhất bảy ca kiểm thử, trong đó ít nhất ba ca là dữ liệu sai. Bắt buộc **đón toàn bộ các lệnh vào một lần `sendall()` duy nhất** rồi mới đọc lần lượt các câu trả lời — đây chính là phép thử ranh giới thông điệp. Chương trình tự in Đạt hoặc SAI cho từng ca; chép kết quả in ra vào Bảng 2.
5. **Hai client và tính tuần tự của server.** Cho client A đăng nhập, gửi một tin rồi QUIT; sau đó client B đăng nhập và gọi HISTORY để đọc lại tin của A. Tiếp theo, trong lúc A đang giữ phiên thì mở B: ghi lại thời điểm `connect()` của B thành công và thời điểm B nhận được câu trả lời đầu tiên. Mô tả trong 3 – 5 dòng vì sao hai mốc này cách nhau.

Bảng 2. Mẫu bảng ca kiểm thử của `test_server.py` (dòng đầu là ví dụ)

TT	Mục đích	Lệnh gửi	Trả lời mong đợi	Trả lời thực tế / Đạt
1	Chặn sai trình tự	MSG all chào	ERR 3 Chưa đăng nhập	ERR 3 Chưa đăng nhập — Đạt
2	Tên rỗng	LOGIN	ERR 2 Tên không hợp lệ	
...	...	...	...	...

2.3. Phần C – Ranh giới thông điệp và truyền tệp (1,5 điểm)

Phần này làm rõ vì sao giao thức dòng văn bản ở Phần A không dùng được cho dữ liệu nhị phân, và bổ sung cách đóng khung bằng tiền tố độ dài vào `protocol.py`.

1. **Thí nghiệm dính gói – tách gói.** Dùng khung mã gợi ý bên dưới: phía gửi gửi ba thông điệp ngắn liên tiếp rồi một khối 5 MB, phía nhận ngủ 1 giây trước khi đọc. **Chạy ba lần** và ghi vào Bảng 3: 60 byte đầu tiên đọc được (cho thấy ba thông điệp đã dính vào nhau hay chưa) và tổng số lần `recv()` để nhận hết khối 5 MB. Nhận xét trong 3 – 5 dòng vì sao ba lần chạy cho ba kết quả khác nhau.
2. **Truyền tệp `file_transfer.py`.** Bổ sung vào `protocol.py` các hàm `recv_exactly()`, `send_msg()` và `recv_msg()` dùng tiền tố độ dài 4 byte (`struct.Struct("!I")`) kèm giới hạn `MAX_MSG`. Dùng chúng để gửi một tệp có **tên tiếng Việt có dấu** và kích thước ít nhất 1 MB. Phía nhận bắt buộc gọi

`os.path.basename()` trên tên tệp nhận được; giải thích trong 2 – 3 dòng vì sao. Đối chiếu kích thước và mã băm `hashlib.sha256` hai đầu, ghi vào Bảng 4.

3. **Chọn cách đóng khung.** Lập bảng so sánh bốn cách đóng khung đã học (kết thúc bằng EOF, độ dài cố định, ký tự phân cách, tiền tố độ dài) theo ba cột: ưu điểm – nhược điểm – khi nào dùng. Sau đó ghi rõ nhóm chọn cách nào cho lệnh chat, cách nào cho nội dung tệp, và bổ sung lựa chọn đó vào bản đặc tả ở Phần A.

Khung gợi ý cho thí nghiệm đính gói – tách gói (hai tệp nhỏ, chạy hai cửa sổ):

```
# --- phía_nhan.py: bận 1 giây rồi mới đọc ---
conn, addr = srv.accept()
time.sleep(1)
dau = conn.recv(60)          # (1) đọc tối đa 60 byte
print(dau)                  # ba tin có dính nhau?
tong, lan = len(dau), 1
while True:
    chunk = conn.recv(65536)
    if not chunk:            # bên gửi đã đóng
        break
    tong += len(chunk); lan += 1
print(tong, "byte /", lan, "lần recv()") # (2) tách gói

# --- phía_gui.py ---
for i in range(1, 4):
    s.sendall(f"MSG|all|tin {i}\n".encode("utf-8"))
s.sendall(b"x" * 5_000_000) # rồi đóng socket
```

Bảng 3. Mẫu bảng thí nghiệm đính gói – tách gói

Lần chạy	60 byte đầu tiên đọc được (dán nguyên văn)	Ba thông điệp có dính nhau không?	Số lần <code>recv()</code> để nhận hết 5 MB
1			
2			
3			

Bảng 4. Mẫu bảng kiểm tra truyền tệp

Tên tệp gửi	Kích thước (byte)	SHA-256 bên gửi (8 ký tự đầu)	SHA-256 bên nhận (8 ký tự đầu)	Khớp

2.4. Phần D – Câu hỏi thảo luận (1,0 điểm)

Nhóm thảo luận và trả lời ngắn gọn (mỗi câu 5 – 10 dòng) trong báo cáo, có dẫn chứng từ kết quả chạy thực tế của chính nhóm ở các phần trên:

- Câu 1. Vì sao câu trả lời HISTORY phải báo trước số dòng? Nếu server chỉ gửi các dòng rồi im lặng thì client gặp chuyện gì? Nêu hai cách khác để đánh dấu điểm kết thúc của một câu trả lời nhiều dòng.

Câu 2. Client gửi 10 MB rồi mới đọc, server vừa đọc vừa gửi lại — vì sao cả hai cùng treo? Giải thích bằng bốn bộ đệm và bằng số liệu ở Bảng 3 của nhóm. Nêu hai cách thiết kế giúp tránh được bế tắc này.

Câu 3. `connect()` của client thứ hai vẫn thành công ngay trong khi server đang bận với client thứ nhất — vì sao? Tham số 5 trong `listen(5)` có ý nghĩa gì? Dẫn chứng bằng quan sát ở mục 5 của Phần B.

3. Sản phẩm nộp và quy cách

- **Báo cáo:** một tệp PDF xuất từ Word / Google Docs (không nộp ảnh quét), đặt tên `NhomZZ_Baitap3.pdf`, trong đó ZZ là mã nhóm. Cấu trúc bắt buộc, dùng đúng tên mục: Trang bìa (số nhóm; họ tên và mã sinh viên của mọi thành viên; công nhóm sử dụng); Phần A (Bảng 1 đã điền, bảng mã lỗi, sơ đồ trạng thái, log server); Phần B (bảng tình huống lỗi của client, Bảng 2 đã điền, nhận xét về hai client); Phần C (Bảng 3 và Bảng 4 đã điền, bảng so sánh bốn cách đóng khung); Phần D; Công cụ AI đã dùng (nếu có); Tài liệu tham khảo; Phụ lục A – Bảng phân công. Mọi số liệu quan sát được phải ghi thành bảng hoặc câu văn; ảnh chụp chỉ để minh họa, cắt gọn phần cần xem, đọc được chữ và có chú thích (Hình 1, Hình 2...). Báo cáo nộp phải là bản hoàn chỉnh, không còn ô trống hay ghi chú chờ điền của mẫu.
- **Mã nguồn:** năm tệp `protocol.py`, `chat_server.py`, `chat_client.py`, `test_server.py`, `file_transfer.py` đặt ở thư mục gốc của tệp nén `NhomZZ_Baitap3_code.zip`; ảnh chụp kết quả đặt trong thư mục `anh/` và đặt tên theo mục (ví dụ `A5_ncat.png`, `B5_hai_client.png`, `C1_lan1.png`); không nén thư mục `.venv`, `__pycache__`, không nén tệp dùng để thử truyền. Mỗi tệp mã có chú thích đầu tệp ghi tên nhóm, thành viên viết tệp đó và lệnh chạy.
- **Chạy được bằng đúng lệnh ghi ở đầu tệp:** giảng viên chạy thử trên một máy với `127.0.0.1` và cổng của nhóm, bằng Python 3.12 trở lên, không cài thêm thư viện. Ví dụ: `python chat_server.py --port 9005` và `python chat_client.py --host 127.0.0.1 --port 9005`. Mã không chạy được bằng lệnh đã ghi bị tính là chưa hoàn thành phần tương ứng.
- **Thành viên:** danh sách ở trang bìa và Phụ lục A phải khớp với nhóm trên e-Learning tại thời điểm nộp; nhóm có thay đổi thành viên sau khi nộp phải nộp lại báo cáo trước hạn.
- **Cách chấm:** không phải làm slide và không thuyết trình. Giảng viên chấm trên báo cáo PDF theo rubric ở mục 4, đối chiếu với ảnh trong tệp nén, chạy thử mã nguồn của nhóm và có thể hỏi bất kỳ thành viên nào tại buổi thực hành.
- **Nơi nộp:** mục "Thực hành 3" trên e-Learning (nhhai.net); nhóm trưởng nộp một lần cho cả nhóm; nộp lại thì thay cả hai tệp.

4. Tiêu chí đánh giá (Rubric R1.2)

Tiêu chí	Điểm	Tốt (80 – 100%)	Đạt (50 – 79%)	Chưa đạt (< 50%)
A. Server chat tuần tự	3,5	Đủ bốn lệnh đúng đặc tả; mã tách hàm rõ ràng; có tham số dòng lệnh, log và dừng an toàn; lỗi của một client không làm chết server.	Chạy được LOGIN và MSG; còn thiếu HISTORY hoặc phần log, tham số dòng lệnh, dọn dẹp trạng thái.	Không chạy được bằng lệnh đã ghi, hoặc không theo đặc tả của nhóm, hoặc chết khi gặp dữ liệu sai.
B. Client và bộ kiểm thử	3,0	Client giữ kết nối bền vững, đọc đúng trả lời nhiều dòng, báo lỗi rõ ràng; bộ kiểm thử đủ bảy ca, gửi dồn một lần và tự báo Đạt / SAI.	Client chạy được trường hợp thông thường; bộ kiểm thử còn thiếu ca lỗi hoặc không gửi dồn một lần.	Client treo hoặc lệch nhịp khi đọc trả lời nhiều dòng; không có bộ kiểm thử chạy được.
C. Ranh giới thông điệp và truyền tệp	1,5	Đủ số liệu ba lần chạy và giải thích đúng; truyền tệp khớp mã băm; chọn cách đóng khung có lý lẽ.	Có thí nghiệm hoặc truyền tệp nhưng thiếu số liệu, thiếu mã băm hoặc giải thích sơ sài.	Không có số liệu chạy thật; truyền tệp hỏng hoặc không dùng tiền tố độ dài.
D. Câu hỏi thảo luận	1,0	Trả lời đúng cả ba câu, có lập luận và dẫn chứng từ kết quả chạy của nhóm.	Trả lời đúng hướng nhưng còn chung chung hoặc thiếu một câu.	Trả lời sai hoặc bỏ trống từ hai câu trở lên.
E. Hình thức báo cáo, mã nguồn và tài liệu tham khảo	1,0	Trình bày đúng quy cách; mã nguồn sạch, có chú thích; trích dẫn ít nhất ba tài liệu tin cậy.	Đúng quy cách; mã nguồn còn lộn xộn hoặc trích dẫn chưa đúng chuẩn.	Sai quy cách; mã nguồn không chú thích; không có tài liệu tham khảo.

5. Quy định

- Báo cáo hoặc mã nguồn sao chép giữa các nhóm, hoặc sao chép từ nguồn khác mà không trích dẫn, nhận 0 điểm cho tất cả các nhóm liên quan.
- Được phép dùng công cụ AI để hỗ trợ tra cứu, sửa lỗi, nhưng phải ghi rõ trong mục "Công cụ AI đã dùng" của báo cáo: công cụ nào, dùng cho việc gì; không dán ảnh chụp giao diện của công cụ AI vào báo cáo; mọi thành viên phải hiểu và giải thích được nội dung nộp khi giảng viên hỏi.
- Thành viên không tham gia (theo bảng phân công có xác nhận của nhóm) hoặc không có tên trong báo cáo nhận 0 điểm bài tập này.
- Sau khi chấm, giảng viên có thể yêu cầu nhóm demo trực tiếp sản phẩm và trả lời câu hỏi về phần việc của từng thành viên; điểm có thể được điều chỉnh tăng hoặc giảm theo kết quả demo, cho cả nhóm hoặc riêng từng thành viên.

- Chỉ chạy chương trình trong mạng của phòng thực hành và trên máy của chính mình. Không quét cổng, không kết nối tới máy của nhóm khác khi chưa được đồng ý. Tập dùng để thử truyền phải là tập của chính nhóm, không chứa dữ liệu cá nhân của người khác.

6. Tài liệu tham khảo và gợi ý

- Slide bài giảng Chương 3 – Lập trình ứng dụng hướng kết nối TCP (mục 3.1 cho đặc tả giao thức và máy trạng thái, mục 3.2 cho khung server, mục 3.3 cho khung client, mục 3.4 cho ranh giới thông điệp và protocol.py).
- B. Rhodes, J. Goerzen – Foundations of Python Network Programming, 3rd ed., Apress, 2014: Chương 3 (TCP) và Chương 5 (Network Data and Network Errors).
- J. Kurose, K. Ross – Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021: mục 2.7 (Socket Programming: Creating Network Applications).
- Python Software Foundation – The Python Standard Library: socket, struct, json, argparse, logging – <https://docs.python.org/3/library/socket.html>
- Real Python – Socket Programming in Python (Guide) – <https://realpython.com/python-sockets/>
- RFC 5321 (SMTP) và RFC 1939 (POP3) – ví dụ về giao thức dòng văn bản có mã trả lời và câu trả lời nhiều dòng – <https://www.rfc-editor.org/rfc/rfc1939>

Phụ lục A – Mẫu bảng phân công công việc và tự đánh giá

Ghi đủ mọi thành viên của nhóm. Cột "Công việc đảm nhận" ghi cụ thể phần đã làm (ví dụ: Phần A – handle_request và bảng mã lỗi; Phần C – file_transfer.py; tổng hợp báo cáo). Cột "Đóng góp (%)" do cả nhóm thống nhất — làm ngang nhau thì ghi bằng nhau (nhóm 5 người: 20 % mỗi bạn); nhóm chịu trách nhiệm về bảng này khi giảng viên hỏi.

TT	Họ và tên	Mã sinh viên	Công việc đảm nhận	Đóng góp (%)
1			Nhóm trưởng; ...	
2				
3				
4				
5				